

Admin Réseaux

Notes de cours

Table des matières

1	Rappels commandes Linux	2
1.1	Structure d'une commande Bash	2
1.2	Commandes de base	2
2	Permissions Linux	3
2.1	Lecture des permissions	3
2.2	Permissions en octet	3
2.3	chmod — changer les permissions	3
3	Scripts Bash	4
3.1	Généralités	4
3.2	Variables	4
3.3	PAM	4
3.4	Pipe et redirections	4
3.5	Horodater un nom de fichier	4
3.6	Code de retour	4
3.7	stdin, stdout, stderr	5
3.8	grep et expressions régulières	5
3.9	ping	5
3.10	Fonctions bash	5
3.11	Conditions (if)	5
3.12	Boucle while	6
3.13	Paramètres d'un script	6
4	TP du 17/04 — Informations	6
5	Sauvegarde	7
5.1	Règle 3-2-1-0	7

1 Rappels commandes Linux

1.1 Structure d'une commande Bash

Une commande Bash est constituée de :

- un **programme**
- un **argument**
- une **option**

Exemple :

```
ls /etc/apache2 -lah
ls -lah /etc/apache2    % equivalent, peuvent etre interchanges
```

- `ls` : fait appel à un programme
- `/etc/apache2` : argument du programme
- `-lah` : une option

Si on met juste le programme, a-t-on un argument par défaut ?

Oui, par défaut — pour `ls` c'est `.` (répertoire courant).

1.2 Commandes de base

<code>ls -lah</code>	Lister les fichiers (human-readable, all, droits)
<code>cd</code>	Changer de répertoire (<i>change directory</i>) Sans argument : revient à la racine utilisateur courant (<code>\$HOME</code>)
<code>cat</code>	Concaténer / afficher le contenu d'un fichier
<code>mkdir</code>	Créer un répertoire (<i>make directory</i>) <code>-p</code> : crée les dossiers parents si nécessaire
<code>pwd</code>	Donne le chemin absolu courant
<code>passwd</code>	Modifier le mot de passe
<code>rm</code>	Supprimer (<i>remove</i>) <code>rm -Rf /*</code> (à ne surtout pas faire !) — <code>-R</code> récursif, <code>-f</code> force
<code>touch</code>	Créer un fichier vide
<code>nano</code>	Éditeur de texte
<code>cp <src> <dest></code>	Copie de fichier ; <code>-R</code> pour récursif
<code>scp <src> <dest></code>	Idem mais via SSH
<code>mv <src> <dest></code>	Déplacer un fichier
<code>clear</code>	Vider le buffer / nettoyer le terminal
<code>usermod</code>	Ajouter un utilisateur dans un groupe
<code>useradd / adduser</code>	Créer un utilisateur

```
groupadd / addgroup      Créer un groupe
sudo                    Substitute User Do — par défaut l'utilisateur de substitution est root
su                      Substitute User : connexion temporaire à un autre utilisateur
                        su - : root
                        su <nom_utilisateur>
```

2 Permissions Linux

2.1 Lecture des permissions

Exemple de sortie `ls -l` :

```
-rw- rw- --x   root   root   script.sh
```

- 1^{er} caractère : type d'élément
 - **d** : directory
 - **-** : fichier brut
 - **l** : lien symbolique
- Colonnes suivantes (par groupe de 3) : **utilisateur propriétaire** | **groupe propriétaire** | **autres utilisateurs**

2.2 Permissions en octet

Permission	Valeur
r (read)	4
w (write)	2
x (execute)	1
0 (rien)	0

2.3 chmod — changer les permissions

```
chmod 0554 script.sh
      ^  groupe
      type d'elt
```

Option : **-R** pour appliquer récursivement.

Bonne pratique : ne jamais donner **w** et **x** simultanément — risque d'injection de code malveillant et d'exécution.

```
chmod 0331 truc.exe    % DANGEREUX : ecriture + execution
chmod 0510 truc.exe    % ok
chmod +x   truc.exe    % A bannir (trop permissif)
```

Pour faire face : accorder soit **w** soit **x**, **jamais les deux**.

3 Scripts Bash

3.1 Généralités

Un script bash est une suite de commandes/instructions. Il permet de **limiter les erreurs humaines**.

Règle : on ne fait JAMAIS les MAS (Mises À jour planifiées) directement — il faut tester avant.

La **première ligne** doit toujours être le *shebang* :

```
#!/bin/bash
```

Un script peut être rendu exécutable avec **chmod** ou lancé via :

```
bash script.sh
```

3.2 Variables

```
var1="Hello"
```

En bash, les variables n'ont pas de type.

3.3 PAM

PAM dans l'environnement Linux : *Pluggable Authentication Module* — plateforme d'auto-identification.

3.4 Pipe et redirections

```
cat test.txt | tail -n k
```

Le **pipe** (`|`) permet d'envoyer le résultat d'une commande dans une autre (« tuyau »). On peut enchaîner autant de pipes qu'on veut.

- > Redirige stdout vers un fichier
- 2> Redirige stderr vers un fichier
- &> Redirige stdout ET stderr vers un fichier

3.5 Horodater un nom de fichier

```
touch "truc-$(date +%y%m%d%H%M).txt"
```

3.6 Code de retour

```
echo $?
```

- Retour = 0 : commande bien exécutée, sans erreur
- Retour $\neq$ 0 : erreur

Si code erreur = 1 et erreur = 2, le problème est une couche au-dessus.

3.7 stdin, stdout, stderr

- `stdin` : entrée standard
- `stdout` : sortie standard
- `stderr` : sortie d'erreur

3.8 grep et expressions régulières

`grep` affiche toute la ligne contenant le mot cherché.

Utilisation avec Regex (option `-E`) :

```
grep -Eo '([0-9a-zA-Z]{1,3}\.){3}'
^ valeurs cherchees
nb de car. par bloc
separateur
nb de fois
```

L'option `-o` n'affiche que ce qui est entre guillemets (la correspondance).

On peut ajouter une autre expression régulière.

3.9 ping

Sous Linux, `ping` ne s'arrête pas tout seul.

```
ping ... -c n & # -c n : nombre de requetes, & : arriere-plan
ping ... -c n | grep "suite de caracteres"
```

3.10 Fonctions bash

```
nom() {
    echo "hello" $1 # $1 : 1er parametre
}

nom "truc en plus" # affiche : hello truc en plus
```

3.11 Conditions (if)

```
if [ x condition x ]; then # espaces obligatoires entre crochets
    instruction
else / elif
    instruction
fi
```

Opérateurs de comparaison :

Opérateur	Signification
<code>-eq</code>	equal
<code>-lt</code>	lower than
<code>-gt</code>	greater than

Négation d'une condition :

```
if [ ! -d $dossier ]    # -d : est-ce un dossier ?
```

3.12 Boucle while

```
while [ condition ]; do
    instruction
done

sleep 2    # attendre 2 secondes
```

3.13 Paramètres d'un script

```
# exemple.sh
echo "des trucs" $1 $2

# Appel :
./exemple.sh machin bidule
# Affiche : des trucs machin bidule
```

4 TP du 17/04 — Informations

- Groupe et identifiant de l'utilisateur courant :
 - uid pour l'utilisateur
 - gid pour le groupe
- Date sur 2 lignes différentes
- Nombre de fichiers dans un dossier : via pipe
- Horodater le fichier archivé

Commandes utiles :

```
head -n -X          # supprime les X premières lignes
ls -lah             # 40k = taille totale du repertoire courant (-l)
watch ls -l chemin  # voir les changements dans le dossier en
                    direct
ls truc 2> log.log   # écrit les codes d'erreur vers log.log
```

Indication devoir : ≈ 83 lignes

Dossier de sortie : /tmp — fichier log

En cas d'échec : réessayage avec un nombre limité d'essais

Rendu : le fichier .sh

5 Sauvegarde

Outil Linux pour la sauvegarde :

- **dd** : copie bit à bit (bytes ou bits)
- **Rsync** : sauvegarde fichiers

Sauvegarde différentielle Se base sur la sauvegarde complète

Sauvegarde incrémentielle Se base sur la dernière sauvegarde

5.1 Règle 3-2-1-0

Chiffre	Signification
3	3 sauvegardes
2	2 supports différents
1	1 hors site
0	0 erreur de restauration

La sauvegarde doit être **immuable** (ne peut pas être modifiée) et testée régulièrement.